Credit Name: CSE 3130 Object-Oriented Programming 2
Assignment Name: Account, PersonalAcct, BusinessAcct

Understanding the Problem
How did you approach understanding the challenge?
Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

I understood the problem completely except for the part where it mentioned that money would be withdrawn from the account if it went below a certain threshold. I wasn't sure if it charged you if you made any transaction (withdrawing and depositing) or just withdrawing. I ended up asking Mr. Abdalla for clarification.

Planning the Solution
Did you create a plan or break the problem into smaller steps before coding?
How did you decide on the tools, data structures, or algorithms to use?

Because the Account and Customer classes that were provided came with instructions, I didn't feel the need to make a concrete plan. Instead, I decided to follow said instructions and figure it out as I went.

Implementation
Did you write the code in small pieces or attempt the entire solution at once?
How did you test your solution along the way to make sure it was working?

I started by downloading the Account and Customer classes that were given in D2L and following the instructions that were given to me. For the Account class, it mostly came down to me adding the parameters to store information about the street, city, province, and postal code. Along with that, a method was needed to change the address information.

```
public Account(double bal, String fName, String lName, String s, String c, String p, String pCode)
```

```
public void changeAddress(String s, String c, String p, String pCode)
{
    cust.changeStreet(s);
    cust.changeCity(c);
    cust.changeProvince(p);
    cust.changePostalCode(pCode);
}
```

A similar thing was done in the Customer class.

```
        street = s;
        city = c;
        province = p;
        postalCode = pCode;
    }

    //create changeStreet method that asks the
    public void changeStreet(String s)
    {
        street = s;
    }

    //create changeCity method that asks the us
    public void changeCity(String c)
    {
        city = c;
    }

    //create changeProvince method that asks th
    public void changeProvince(String p)
    {
        province = p;
    }

    //create changePostalCode method that asks
    public void changePostalCode(String pCode)
    {
        postalCode = pCode;
    }
```

The next thing I did was I created the PersonalAcct and BusinessAcct classes. In them, I used the extends keyword in order to inherit information from the Account class.

```
public PersonalAcct(double bal, String fName, String lName, String s, String c, String p, String pCode)
{
    super(bal, fName, lName, s, c, p, pCode);
}
```

I then made the Bank class to test the code. Once again, I recycled code from the UEmployee class as a baseline.

```
PersonalAcct acct1 = new PersonalAcct(100, "Calen", "Plana", "1", "Calgary", "Alberta", "A1B 2C3");
BusinessAcct acct2 = new BusinessAcct(500, "Calen", "Plana", "6", "Edmonton", "Alberta", "E4F 5G6");

NumberFormat money = NumberFormat.getCurrencyInstance();
Scanner input = new Scanner(System.in);

String choice, street, city, province, postalCode;
int select;
double amount;

Account acct = acct1;
```

After that, I wrote the code that deposits money and the code for changing the user's address since they were fairly straightforward.

```
else if (choice.equalsIgnoreCase("W"))
{
    System.out.println("Enter Amount You Would Like to Withdraw: ");
    amount = input.nextDouble();
    acct.test(amount);
}
```

```
else if (choice.equalsIgnoreCase("C"))
{
    System.out.println("Please Enter Your New Street: ");
    street = input.next();

    System.out.println("Please Enter Your New City: ");
    city = input.next();

    System.out.println("Please Enter Your New Province: ");
    province = input.next();

    System.out.println("Please Enter Your New Postal Code: ");
    postalCode = input.next();

    acct.changeAddress(street, city, province, postalCode);
    System.out.println("Your Account Information is Now: " + acct.toString());
}
```

The last thing that was required was to write the code for withdrawing money from the accounts. To do this, I went back to the Account class to slightly modify the withdrawal method and to make an abstract method. Then I went to the PersonalAcct and BusinessAcct to write their respective abstract methods that connect back to an account that contains the logic for when the accounts go below the minimum balance threshold.

```
void test(double amt)
{
    double balance;

    balance = super.getBalance();

    if (amt <= balance)
    {
        super.withdrawel(amt);
    }

    else
    {
        System.out.println("Not enough money in account.");
    }

    if (super.getBalance() < 100)
    {
        super.withdrawel(2);
        System.out.println("Your Account Has Gone Below the Minimum, and $2 Have Been Charged.");
    }

    System.out.println("Your Balance is Now: " + money.format(super.getBalance()));
}
```

Overcoming Challenges

What part of the problem was the most difficult for you?
How did you handle moments when you felt unsure of what to do next?

The only difficult part was understanding the instructions that described what the program does since it wasn't very clear. To solve this problem, I just had to clarify with Mr. Abdalla.

Learning

Was there anything you learned that you think will help you with future challenges?

My understanding of inheritance and abstract methods have improved.